



Basic Cloud Computing

PROJECT REPORT

Silvia Calabretta - SM3200087

March 25, 2026

Contents

1	Introduction	1
2	Virtual Machine Cluster Setup	1
2.1	Template VM	1
2.2	Network Setup	2
2.3	Master Node Configuration	2
2.3.1	DNS with dnsmasq	3
2.3.2	Shared Filesystem with NFS	6
2.4	Worker Nodes Setup	6
2.4.1	Network Configuration	7
2.4.2	Shared Filesystem	8
2.4.3	Automount Configuration	8
2.4.4	SSH Configuration	9
2.5	Replication of Worker Nodes	9
2.6	Conclusion	10
3	Containers Cluster Setup	11
3.1	Architecture	11
3.1.1	Docker Compose file	11
3.1.2	Dockerfile	12
3.1.3	start.sh	12
3.1.4	Benchmarks	13
3.2	Container Deployment	13
4	Benchmarks	14
4.1	Benchmarks and Experimental Setup	14
4.2	Benchmark Configuration	14
4.3	Execution on Virtual Machines	16
4.3.1	HPCC Configuration	17
4.4	Execution on Containers	17
5	Results	18
5.1	CPU results	18
5.2	Memory results	18
5.3	Disk results	19
5.4	Network results	20
5.5	HPCC	21
6	Technical Challenges	22

6.1	Benchmark tool selection and stress-ng exclusion	22
6.2	HPC Challenge: Stability and Virtualization Failures	23
7	Conclusion	24
	List of Figures	I
	Nomenclature	I
	References	II

1 Introduction

The rise of cloud computing has intensified the debate between traditional virtualization and containerization for high-performance workloads. The aim of this project is to evaluate the performance trade-offs between Virtual Machines (VMs) and Containers when deployed in a clustered environment.

To achieve this, we built two identical clusters, each consisting of three nodes: one master and two workers. To ensure a fair comparison, each node was provisioned with 2 vCPUs and 2048 MB of RAM. The project was developed on an ASUS Vivobook with Windows 11. The VM cluster was built using VirtualBox, a standard type 2 hypervisor, while the container cluster leveraged the Docker engine running on the Windows Subsystem for Linux 2 (WSL 2) backend.

The performance of these clusters was evaluated through a comprehensive benchmarking suite, including Sysbench, IOZone, Iperf3 and the HPC Challenge (HPCC).

This report details the experimental setup, analyzes the resulting performance metrics, and discusses the stability challenges encountered under high-stress scenarios.

2 Virtual Machine Cluster Setup

The first part of the project focused on setting up a cluster of Linux virtual machines (VMs). The goal was to configure a master node and multiple worker nodes so that they could connect both to the internet and to each other through an internal private network. Additionally, the setup included configuring SSH access, IPs, DNS, a shared filesystem, and a simple load-balancing service.

For this purpose, we rely on VirtualBox. The referenced tutorial¹ has been a valuable guide throughout this process.

The environment was prepared with:

- VirtualBox installed on the Windows host machine;
- Ubuntu 24.04.2 amd64 ISO image (matching the architecture of the host machine);
- an SSH client for connecting to the VMs.

2.1 Template VM

A base template VM was created with:

- 2 vCPU, 2048MB RAM, 25 GB disk,
- a NAT adapter for internet access,
- credentials (username: user01, password: test).

¹<https://github.com/Foundations-of-HPC/Cloud-basic-2024/blob/main/Tutorials/VirtualMachine/README.md>

Ubuntu 24.04 LTS was installed on this VM, with OpenSSH server enabled. After installation, the system was updated and useful packages such as `net-tools`, `gcc`, and `make` were also installed.

This VM served as the base image, that was cloned to create one master node (`master`) and two worker nodes (`node01`, `node02`).

2.2 Network Setup

A host-only network named `VirtualBox Host-Only Ethernet Adapter` (Windows makes it extremely difficult to rename it) was configured with:

Host IPv4 Address: 192.168.56.200

IPv4 Network Mask: 255.255.255.0

DHCP Server Address: 192.168.56.201

DHCP Server Range: 192.168.56.2 - 192.168.56.199

The IP address 192.168.56.1 is left available for the master node.

Each VM was equipped two adapters:

1. NAT for internet access with IPv4 10.0.2.15 (tested with the command `ping google.com`);
2. Host-Only Adapter to communicate with the host and the other VMs (the `VirtualBox Host-Only Ethernet Adapter` just created).

Port forwarding rules using TCP were added in the NAT networking, to forward SSH traffic from specific ports on the host (3022 (`master`), 4022 (`node01`), 5022 (`node02`)) to guest's port 22 (the default SSH port).

2.3 Master Node Configuration

In order to facilitate ssh connection from the host machine to the master node, an ssh key was generated on the host and a copy of the public key was added to the folder `~/.ssh/authorized_keys` on the master VM.

Now we want to configure the IP addresses. As expected, the following network interfaces were shown:

```
1  $ ip link
2
3  1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
   DEFAULT group default qlen 1000
4  link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
   UP mode DEFAULT group default qlen 1000
6  link/ether 08:00:27:10:d9:39 brd ff:ff:ff:ff:ff:ff
```

```
7 3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode
   DEFAULT group default qlen 1000
8  link/ether 08:00:27:f6:13:9a brd ff:ff:ff:ff:ff:ff
```

enp0s3 is the primary interface connected to the NAT, while enp0s8 is the secondary interface connected to the host-only network.

In order to set a static IP for the host-only adapter, we modify the netplan configuration in the 50-cloud-init.yaml file:

```
1 $ sudo vim /etc/netplan/50-cloud-init.yaml
2
3 network:
4   version: 2
5   ethernets:
6     enp0s3:
7       dhcp4: true
8     enp0s8:
9       dhcp4: false
10    addresses: [192.168.56.1/24]
```

The hostname was set to master.

2.3.1 DNS with dnsmasq

The /etc/hosts file was updated by adding the following addresses for the other nodes:

```
1 $ sudo vim /etc/hosts
2
3 127.0.0.1 localhost
4 127.0.1.1 master
5 192.168.56.1 master
6
7 192.168.56.2 node01
8 192.168.56.3 node02
9 192.168.56.4 node03
10 192.168.56.5 node04
11 192.168.56.6 node05
12 192.168.56.7 node06
13 192.168.56.8 node07
14 192.168.56.9 node08
15 192.168.56.10 node09
16
```

```
17 # The following lines are desirable for IPv6 capable hosts
18 ::1      ip6-localhost ip6-loopback
19 fe00::0  ip6-localnet
20 ff00::0  ip6-mcastprefix
21 ff02::1  ip6-allnodes
22 ff02::2  ip6-allrouters
```

We can now proceed with the configuration of dnsmasq to provide DHCP and DNS services for the cluster. We will be working on three key configuration files:

- /etc/resolv.conf is the system DNS configuration file used by all applications.
- /etc/dnsmasq.conf is the main configuration file for dnsmasq. It defines how dnsmasq handles DNS queries, DHCP assignments, caching, logging, and which network interfaces it listens to.
- /etc/resolv.dnsmasq specifies which external DNS servers dnsmasq should use for queries that are not local.

In order to not lose the default configuration file, we make a copy in `/etc/dnsmasq.conf.ORG` and then modify `dnsmasq.conf`.

```
1  $ sudo apt install dnsmasq
2  $ sudo cp -ip /etc/dnsmasq.conf /etc/dnsmasq.conf.ORG
3  $ sudo vim /etc/dnsmasq.conf
4
5  bogus-priv
6  bind-interfaces
7  cache-size=1000
8  resolv-file=/etc/resolv.dnsmasq
9  listen-address=127.0.0.1, 192.168.56.1
10 log-queries
11 dhcp-range=192.168.56.2,192.168.56.10,12h
```

The main configuration commands are:

1. **bogus-priv**: prevents forwarding DNS queries for private, non-routable addresses to public servers, protecting the internal network.
2. **bind-interfaces**: restricts dnsmasq to the addresses specified in `listen-address`, avoiding responses from unwanted interfaces.
3. **cache-size=1000**: sets the DNS cache to 1000 entries, speeding up repeated queries.
4. **resolv-file=/etc/resolv.dnsmasq**: defines the DNS servers to which unresolved queries are forwarded.
5. **listen-address=127.0.0.1, 192.168.56.1**: limits dnsmasq to listen only on localhost and the internal network interface.

6. **log-queries**: enables logging of all DNS requests, useful for debugging and monitoring.

7. **dhcp-range**: assigns IP addresses and hostnames to virtual machines in the host-only network.

Since systemd-resolved manages and overwrites `/etc/resolv.conf`, the original file is removed and replaced:

```
1 $ sudo unlink /etc/resolv.conf
2 $ sudo vim /etc/resolv.conf
3
4 nameserver 127.0.0.1
5 options edns0 trust-ad
6 search .
```

In this way, the system is configured to use dnsmasq itself as the local DNS server (127.0.0.1). This ensures that all DNS requests from the master VM are first handled by dnsmasq, which either resolves local names or forwards unresolved queries to external DNS servers via `/etc/resolv.dnsmasq`.

A symbolic link is now created to connect `/etc/resolv.dnsmasq` to the file managed by systemd-resolved:

```
1 $ sudo ln -s /run/systemd/resolve/resolv.conf /etc/resolv.dnsmasq
```

This way, dnsmasq knows the actual external DNS servers without modifying the system's `/etc/resolv.conf` directly.

Final DNS request flow:

1. DNS local request: is sent to dnsmasq (127.0.0.1 or 192.168.56.1);
2. if the domain is local (e.g. node01), dnsmasq resolves it directly;
3. if the domain is external (e.g., google.com), dnsmasq reads `/etc/resolv.dnsmasq`, which is linked to `/run/systemd/resolve/resolv.conf`, and forwards the request to Internet DNS servers.

In practice, the system never contacts external DNS servers directly, all queries are managed centrally through dnsmasq.

We can now restart the services, in order to apply the changes, and enable dnsmasq to run automatically at system boot:

```
1 $ sudo systemctl restart dnsmasq systemd-resolved
2 $ sudo systemctl enable dnsmasq
3
4 # reboot the system and check the status
5 $ sudo reboot
6 $ sudo systemctl status dnsmasq
```


2.3.2 Shared Filesystem with NFS

We want to create a distributed filesystem accessible from all nodes. In order to do that, we use NFS. A shared directory was created on the master and exported:

```
1  $ sudo apt install nfs-kernel-server
2  $ sudo mkdir /shared
3  $ sudo chmod 777 /shared
4
5  $ sudo vim /etc/exports
6  /shared 192.168.56.0/255.255.255.0(rw,sync,no_root_squash,no_subtree_check
    )
```

Note: with the command `sudo chmod 777 /shared`, the directory becomes readable, writable, and modifiable by anyone. This setting is used here for simplicity, although in a production environment it would typically be considered far too permissive.

The directory `/shared` is exported to all clients within the subnet `192.168.56.0/255.255.255.0` with the following options:

- `rw` grants read and write permissions.
- `sync` enforces synchronous data writes.
- `no_root_squash` preserves root privileges for remote clients.
- `no_subtree_check` disables subtree verification for improved performance.

Now we can enable NFS to run automatically at system boot, restart the server and prepare some directories to share

```
1  $ sudo systemctl enable nfs-kernel-server
2  $ sudo systemctl restart nfs-kernel-server
3
4  $ sudo mkdir /shared/data /shared/home
5  $ sudo chmod 777 /shared/data /shared/home
```

Worker nodes will be able to mount these directories with `nfs-common`.

2.4 Worker Nodes Setup

We can now proceed with the configuration of the nodes. We will be referring to the virtual machine `node01`. The same procedure can be replicated in other nodes. Alternatively, after completing the configuration of the first worker node, it is possible to clone the virtual machine and adjust a small number of parameters (see Section 2.5 for details). This approach is significantly more efficient and therefore recommended.

The goal is to enable host-only network connectivity with dynamic addressing, configure secure shell access, and mount a shared NFS directory across the cluster.

2.4.1 Network Configuration

We start from the configuration of a dynamic IP address. We first ensure to work on the correct adapter by calling:

```
1  $ ip link
2
3  1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode
    DEFAULT group default qlen 1000
4  link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5  2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
    UP mode DEFAULT group default qlen 1000
6  link/ether 08:00:27:10:d9:39 brd ff:ff:ff:ff:ff:ff
7  3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode
    DEFAULT group default qlen 1000
8  link/ether 08:00:27:db:6e:fd brd ff:ff:ff:ff:ff:ff
```

Again, enp0s3 is the primary interface connected to the NAT, while enp0s8 is the secondary interface connected to the Host-Only network.

We can now proceed editing the Netplan configuration file `/etc/netplan/50-cloud-init.yaml`

```
1  $ sudo vim /etc/netplan/50-cloud-init.yaml
2
3  network:
4    version: 2
5    ethernets:
6      enp0s3:
7        dhcp4: true
8        dhcp4-overrides:
9          use-dns: no
10     enp0s8:
11       dhcp4: true
12       dhcp-identifier: mac
13       nameservers:
14         addresses: [192.168.56.1]
15
16  $ sudo netplan apply
```

Notice that for the NAT interface we require not to use the DNS servers provided by the DHCP server. For the host-only adapter instead, we enable the use of DHCP and set the master VM as the DNS server the system should use for resolving domain names.

We now empty the contents of `/etc/hostname`, and update the DNS resolver by unlinking the default `/etc/resolv.conf` and manually specifying the appropriate nameserver:

```
1  $ sudo unlink /etc/resolv.conf
2  $ sudo vim /etc/resolv.conf
3
4  nameserver 192.168.56.1
5  search .
```

A system reboot finalizes these changes, and the assigned addresses can be verified with:

```
1  $ hostname -I
2  10.0.2.15 192.168.56.5 fd00::a00:27ff:fe10:d939
```

2.4.2 Shared Filesystem

In order for the nodes to access the distributed filesystem, the package `nfs-common` must be installed, and a local directory (e.g. `/shared`) created:

```
1  $ sudo apt install nfs-common
2  $ sudo mkdir /shared
```

The remote export from the host (`192.168.56.1:/shared`) is mounted in the local directory `/shared` with:

```
1  $ sudo mount 192.168.56.1:/shared /shared
```

Successful mounting can be verified with `df -h`, and read/write access tested by creating a file within the shared directory. Since the directory is visible to all nodes, the file will appear across the cluster. For better organization of the benchmarking process, we create two subdirectories, `/shared/home` and `/shared/data`. The filesystem can be unmounted with:

```
1  $ sudo umount /shared/
```

2.4.3 Automount Configuration

It is possible to enable automatic mounting using AutoFS. After installing the `autofs` package, the configuration consists in adding an entry in `/etc/auto.master`, telling that the directory `/shared` has to be managed by `autofs` according to an auxiliary map file `/etc/auto.shared`. The latter specifies the mount point and remote location.

```
1  $ sudo apt install autofs
2  $ sudo vim /etc/auto.master
3
4  ...
5  /shared    /etc/auto.shared
6
7  $ sudo vim /etc/auto.mount
8
9  data      192.168.56.1:/shared/data
10 home     192.168.56.1:/shared/home
```

After modifying these files, the service is restarted with:

```
1  $ sudo systemctl restart autofs
```

This configuration ensures that the shared directory is mounted automatically when accessed, thus providing persistent and transparent availability of the shared storage across all cluster nodes.

2.4.4 SSH Configuration

To facilitate passwordless authentication and the following benchmark evaluation, we enable seamless SSH remote access. SSH keys are generated on the master node using `ssh-keygen`. The public key is then transferred to the nodes simply exploiting the shared filesystem and then appended to the `authorized_keys` file within the `.ssh` directory.

2.5 Replication of Worker Nodes

Once the first node is successfully created and configured, additional nodes can be obtained by cloning the virtual machine and assigning each clone a distinct name, in our case `node02`. The cloned machines inherit the full configuration of the original node, only few modifications are required for proper functioning. From the VirtualBox Manager:

- update the port forwarding rule to use another available port (for instance, if `node01` uses port 4022, then to `node02` could be assigned port 5022);
- generate a new MAC address, which can be done from `Settings > Network > Adapter 2 > MAC Address` by selecting the option to generate a random address.

Within the virtual machine itself, the following steps are required:

- generate a new `machine-id`:

```
1  $ sudo rm /etc/machine-id
2  $ sudo systemd-machine-id-setup
```

- generate new SSH host keys:

```
1  $ sudo rm /etc/ssh/ssh_host_*
2  $ sudo ssh-keygen -A
3  $ sudo systemctl restart ssh
```

Notice that neither the hostname nor the IP address needs to be modified, since both are assigned dynamically by the master node.

2.6 Conclusion

The cluster setup successfully established a working environment with:

- master and worker nodes connected via an internal network,
- centralized DNS and DHCP with `dnsmasq`,
- a shared filesystem over NFS.

This environment provides a foundation for running cloud applications and testing distributed workloads.

3 Containers Cluster Setup

This section focuses on the setup of a container environment similar to what was done with virtual machines. To manage the deployment we used Docker, and in particular Docker Compose. This tool allowed to automatically create the containers and connect them via a shared network and a shared Docker volume, in order to facilitate file sharing.

The architecture consists of one master node and two worker nodes (node01 and node02). These are built from a unified Docker image and coordinated through a shared initialization mechanism that enables passwordless SSH communication across the cluster.

3.1 Architecture

The project is organized as follows:

```
containers/
├── setup/
│   ├── compose.yaml      Orchestration definition
│   ├── Dockerfile        Image specification
│   ├── start.sh          Runtime entrypoint script
│   └── benchmarks/       Benchmarking scripts
│       ├── bench_cpu.sh
│       ├── bench_mem.sh
│       ├── bench_disk.sh
│       └── bench_net.sh
└── run_all.sh            Run all benchmarks script
```

Detailed contents of these files can be found in my github page². The following subsections analyze the specific role and configuration of each component.

3.1.1 Docker Compose file

The cluster is defined in the `compose.yml` file. It specifies three services, `master`, `node01`, and `node02`, which instantiate the nodes of the cluster.

All nodes are configured with the same features:

- all nodes are built from the same local `Dockerfile`;
- each container is restricted at 2 CPUs and 2048MB of memory;
- all services are attached to a custom bridge network named `cluster-net`, enabling internal DNS resolution by hostname;

²<https://github.com/silviacalabretta/cloud-computing-project/tree/main>

- a shared volume `cluster-shared` is mounted to `/shared` inside every container, which acts as a shared filesystem for consistent data exchange.

Explicit dependencies are set using `depends_on` to enforce startup order, although the synchronization logic is primarily handled by the entrypoint script.

3.1.2 Dockerfile

The `Dockerfile` defines the software environment for all nodes in the cluster, based on the `ubuntu:22.04` image. The build process follows these primary stages.

1. **Package installation:** all necessary packages for cluster functioning and benchmarking are installed, including `hpcc`, `sysbench`, `iperf3`, `iozone3`, and `openmpi`.
2. **SSH configuration:** the SSH daemon (`sshd`) is installed and configured. We generate a default SSH host key and explicitly configure the SSH client to disable strict host key checking. While this is generally discouraged in production for security reasons, it is essential in this testing environment to prevent connection failures.
3. **Asset deployment:** benchmark scripts are copied into the image and marked as executable.
4. **Entrypoint:** the `start.sh` script is copied into the image and set as the container entrypoint to manage runtime initialization.

3.1.3 start.sh

The `start.sh` script acts as the runtime controller, handling the trust relationships between the master and the worker nodes that cannot be performed during the image build.

The script firstly ensures that the `shared/home` and `shared/data` subdirectories are created within the volume, for consistency with the virtual machines filesystem. Then it implements a different behavior depending on the role of each container.

- **Master node:** after verifying the existence of SSH keys, a new pair is generated if they are missing; the public key (`id_rsa.pub`) is then copied to the shared volume (as `master.pub`). This makes the master's identity available to all other nodes in the cluster.
- **Worker node:** the worker nodes enter a waiting loop, checking for the existence of `master.pub` in the shared directory. This ensures they do not proceed until the master has advertised its identity. Once the key is found, it is appended to the local `authorized_keys` file.

Finally, regardless of the role, the script sets the correct permissions for the SSH directory and launches the SSH daemon in the foreground to keep the container active and ready for remote command execution.

3.1.4 Benchmarks

All the performance measurements except for `hpcc` were carried out using a set of four custom benchmark scripts, located in the `benchmarks/` directory of the project. Each script is dedicated to the evaluation of a specific subsystem of the cluster (see Section 4 for more details).

To simplify the entire testing process, a control script named `run_all.sh` was created. This script is designed to be executed on the host machine and automatically launches all benchmarks on the appropriate containers using `docker exec`. The script sequentially triggers CPU, memory, and disk benchmarks on each node of the cluster, and coordinates the network benchmarks by dynamically assigning server and client roles to the containers.

All benchmark results are stored in the shared directory `/shared/data`, which is mounted on every container and allows the host system to collect and analyze the output files once the execution is completed.

3.2 Container Deployment

The experimental environment was deployed on a Windows host system using Docker Desktop with the WSL 2 backend. Before building the containers, it was necessary to verify that WSL 2 integration was correctly enabled in the Docker Desktop settings. All project files were then copied into the WSL home directory to simplify file management and interaction with the Linux-based toolchain.

The cluster containers were built using the command `docker compose build` and subsequently launched in detached mode with `docker compose up -d`. The correct creation and execution of the containers was verified using `docker ps`.

Further validation was performed by opening interactive shells inside the containers via `docker exec -it <container_name> bash`, which allowed us to confirm the correct hostnames, verify SSH connectivity from the master node to the worker nodes, ensure that the shared filesystem was properly mounted, and check that the benchmark scripts were correctly copied inside each container.

4 Benchmarks

This section describes the benchmarks employed in the experimental evaluation, the design of the benchmarking scripts and the execution procedures adopted for both virtual machines and container-based environments.

4.1 Benchmarks and Experimental Setup

To evaluate system performance across different computing environments, the following benchmarks were used:

- CPU performance: `hpcc`, `sysbench`;
- Memory performance: `sysbench`;
- Disk I/O (local and shared): `IOZone`,
- Network performance: `iperf3`,
- High Performance Computing (HPC) workloads: MPI-based tests using `hpcc`.

The experimental cluster consists of one master node and two worker nodes. All nodes were deployed on a host system running Windows 11 Home 24H2. The host system is equipped with a single physical CPU comprising 10 cores and 12 logical processors, and 7.7 GB of RAM. Each machine was configured with two virtual CPUs and 2048 MB of RAM.

To automate the experimental process, four benchmarking scripts were developed:

- `bench_cpu.sh`: evaluates computational performance using the `sysbench` CPU benchmark;
- `bench_mem.sh`: measures memory throughput and allocation behavior using `sysbench`;
- `bench_disk.sh`: analyzes disk I/O performance on both the local and shared filesystems using `iozone`;
- `bench_net.sh`: evaluates network bandwidth and latency using `iperf3` and `ping`.

Each script executes the selected benchmark with standardized parameters, collects the output, and stores the resulting logs in a shared directory to ensure consistency and reproducibility.

4.2 Benchmark Configuration

For each benchmark, fixed parameters were selected in order to ensure consistency across all executions and environments.

CPU Benchmark

```
1 sysbench —test=cpu —cpu-max-prime=30000 —threads=2 run
```

The parameter `-cpu-max-prime=30000` controls the upper bound of the prime number search and thus the computational workload of the test. The number of threads was set to 2 to match the number of virtual CPUs assigned to each node, ensuring full utilization of the available processing resources without oversubscription.

Memory Benchmark

```
1 sysbench --test=memory --threads=2 --memory-block-size=1M --memory-total-size=20G run
```

Two threads were used to match the CPU configuration of each node. A block size of 1 MB was selected to model large contiguous memory transfers, while the total transfer size of 20 GB guarantees that the benchmark duration is sufficiently long and exceeds the available physical memory, preventing excessive caching effects.

Disk I/O Benchmark

```
1 iotzone -a -I -s 102400 -r 1024 -f filename
```

The `-a` flag enables automatic testing over a range of access patterns. The `-I` option forces direct I/O, bypassing the operating system cache to expose raw storage performance. The test file size was set to 102400 KB (100 MB), while the record size was fixed at 1024 KB (1 MB) to emphasize large sequential and random I/O operations.

The benchmark was executed twice by setting `filename` to both `/tmp/iotzone_local.tmp` and `/shared/home/iotzone_shared.tmp`, allowing comparison between local and shared storage performance.

Network Benchmark

On the receiving node:

```
1 iperf3 -s
```

On the transmitting node:

```
1 iperf3 -c <server_ip> -t 30
2 iperf3 -c <server_ip> -t 30 -R
```

Each transmission test was executed for 30 seconds to ensure stable throughput measurements. Both transmission directions were evaluated by enabling reverse mode using the `-R` flag.

Latency measurements were obtained using:

```
1 ping -c 50 -i 0.2 <server_ip>
```

A total of 50 ICMP packets were sent at intervals of 0.2 seconds in order to compute statistically meaningful latency estimates.

HPC Challenge (HPCC)

The HPCC benchmark was intentionally executed manually on the master node rather than through an automated script, in order to allow greater flexibility during experimentation and to explore the capabilities of each environment. Multiple problem sizes and block configurations were investigated by appropriately modifying the `hpccinf.txt` configuration file, in particular:

```
1 5 # of problems sizes (N)
2 1024 2048 4096 6144 8192 Ns
3 1 # of NBs
4 192 NBs
5 0 PMAP process mapping
6 1 # of process grids (P x Q)
7 2 Ps
8 3 Qs
```

```
1 1 # of problems sizes (N)
2 6144 Ns
3 4 # of NBs
4 64 128 192 256 NBs
5 0 PMAP process mapping
6 1 # of process grids (P x Q)
7 2 Ps
8 3 Qs
```

```
1 mpirun.openmpi -np 6 --hostfile mpi-hostfile hpcc
```

The number of MPI processes was set to 6, corresponding to the total number of virtual CPUs available in the cluster (2 per node). The hostfile explicitly defines process placement across the master and worker nodes.

The following additional flags were required:

- on virtual machines: `-mca btl_tcp_if_include enp0s8` to explicitly select the network interface used for MPI communication;
- on containers: `-allow-run-as-root` to bypass OpenMPI's default security restrictions, since Docker containers execute processes with root privileges by default.

4.3 Execution on Virtual Machines

On each virtual machine, the required benchmarking and development tools were installed:

```
1 $ sudo apt install -y stress-ng sysbench iperf3 iotest3 hpcc gcc g++
   openmpi-bin libopenmpi-dev make
```

The benchmarking scripts were prepared on the host system and transferred to the master node:

```
1 $ scp -r benchmarks user01@master:/home/user01/
```

They were then made executable and, after checking the assigned IP address, they were propagated to the worker nodes:

```
1 $ chmod +x benchmarks/*.sh
2 $ scp -r benchmarks user01@node04:/home/user01/
3 $ scp -r benchmarks user01@node05:/home/user01/
```

After deployment, `bench_cpu.sh`, `bench_mem.sh`, and `bench_disk.sh` were executed independently on each node. Network benchmarks were performed pairwise between the node combinations master–node04 and node04–node05, with each node alternately acting as server and client while testing both transmission directions.

All benchmark outputs were written to the shared filesystem, allowing centralized access from the master node. Finally, the complete dataset was transferred from the master node to the host system for analysis.

4.3.1 HPCC Configuration

After determining the IP addresses assigned to the worker nodes, the MPI host configuration file `mpi-hostfile` was created:

```
1 $ sudo vim mpi-hostfile
2
3 master slots=2
4 node04 slots=2
5 node05 slots=2
```

Correct configuration was verified using:

```
1 $ mpirun -np 3 -mca btl_tcp_if_include enp0s8 --hostfile mpi-hostfile --map
  -by ppr:1:node hostname
```

The reference configuration file `/usr/share/doc/hpcc/examples/_hpccinf.txt` was copied and tuned according to the cluster size. Multiple problem sizes were evaluated but we were not able to find one that allowed to complete the test (see Section 6.2).

4.4 Execution on Containers

Once the integrity of the environment was established, the full testing process was executed from the host system by running the `run_all.sh` script. All benchmark output files were generated inside the shared directory `/shared/data`.

After completion, the results were collected from the master container.

Notice that under typical Linux installations, Docker volumes can be directly accessed from the host filesystem by inspecting their mount point using `docker volume inspect`. However, when using Docker Desktop with the WSL 2 backend, Docker operates inside an internal Linux virtual machine. As a consequence, the volume mount paths exist inside that virtualized environment and are not directly visible from WSL. For this reason, all result files were retrieved by explicitly copying them from inside the master container using `docker cp`.

5 Results

5.1 CPU results

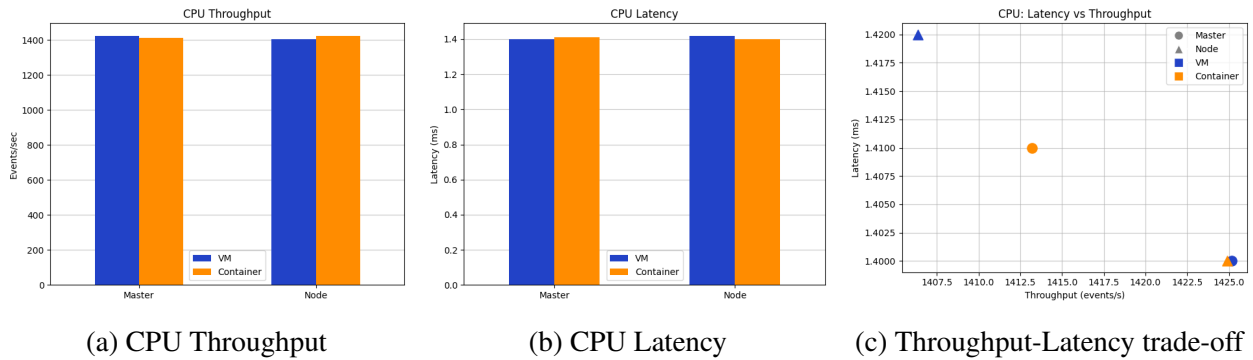


Figure 1: CPU sysbench

Figure 1 presents the results from the sysbench CPU benchmark. Figure 1a shows that both virtual machines and containers perform almost identically, achieving between 1400 and 1425 events per second. This similarity is also seen for the latency, displayed in Figure 1b, where the average processing time stays steady at around 1.40 ms.

Figure 1c offers a correlation of these metrics, comparing them directly. A minor variance in performance is observed in the VM node (shown as the blue triangle), which has a slightly higher latency and lower throughput than the others. However the difference is less than 1.5%, so extremely small.

Overall, the results show that CPU tasks perform with the same efficiency in both scenarios, with variations likely due to minor fluctuations rather than the virtualization technology itself.

5.2 Memory results

The sysbench memory performance metrics are displayed in Figure 2. Unlike the CPU benchmark, Figure 2a reveals a distinct performance gap. Containers significantly outperform virtual machines,

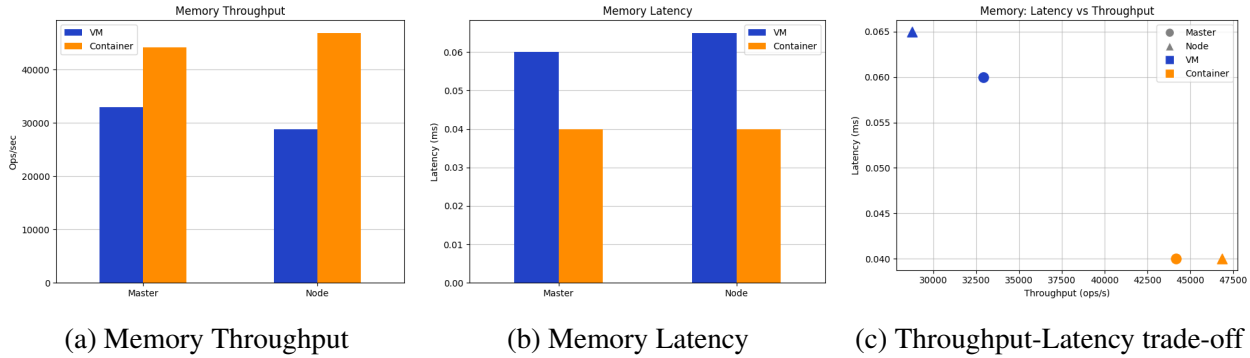


Figure 2: Memory sysbench

achieving about 45,000 operations per second, whereas the virtual machines lagged behind, averaging roughly 30,000 operations per second.

This trend is mirrored in the memory latency plot 2b, where containers maintained a lower, more consistent latency of ≈ 0.04 ms, compared to the ≈ 0.06 ms observed in the VM environment.

Figure 2c clearly visualizes this separation. The blue VM markers cluster in the upper-left, showing lower speed and higher latency, while the orange container markers sit in the bottom-right, showing higher speed and lower latency.

This suggests that, in our setting, containers are much more efficient for memory-intensive operations, as the additional abstraction layer of the VMs introduces noticeable overhead.

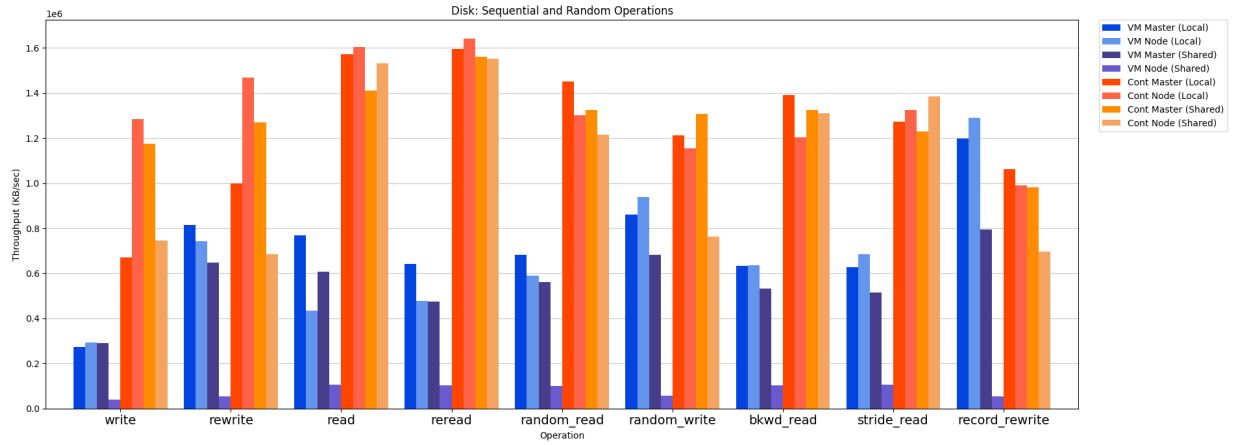
5.3 Disk results

The Disk I/O performance results are presented in Figure 3. The analysis considers both local and shared filesystems. As shown in Figure 3a, containers consistently outperform virtual machines in raw block-level throughput.

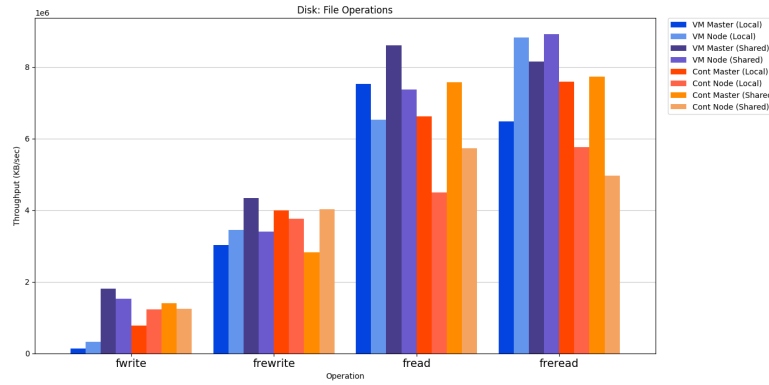
On the local filesystem, containers achieve significantly higher throughput compared to VMs in almost all the operations. But there is an even larger gap of performance on the shared filesystem.

Virtual machines rely on a network mount, through NFS. The master hosts the directory locally, so its shared performance remains relatively high. The nodes, however, must access this data over the virtual network, which leads to very poor performance, particularly for the write operation, that drops to near zero.

In the Docker environment, a shared volume behaves differently. Technically, both the master and node containers are mounting the same physical directory, located on the host's filesystem. There is no network protocol involved: both containers access the shared data via direct bind mounts. Consequently, the nodes suffers no performance penalty compared to the master, maintaining relatively high throughput.



(a) Sequential and random operations



(b) File Operations

Figure 3: Disk I/O IOZone

However, Figure 3b presents a contrasting trend. Here, the throughput scales up significantly (peaking at ≈ 9 GB/s), and virtual machines performance match or exceed the containers' one, particularly in `fread` and `freread`. This is probably due to the memory management occurring with file operations, which is more effective for virtual machines.

Overall, the results demonstrate that, under identical resource constraints, containers provide consistently better disk I/O performance and higher efficiency, particularly for read-dominated and random-access workloads.

5.4 Network results

The network performance results are presented in Figure 4, which reveals an impressive difference between the two technologies.

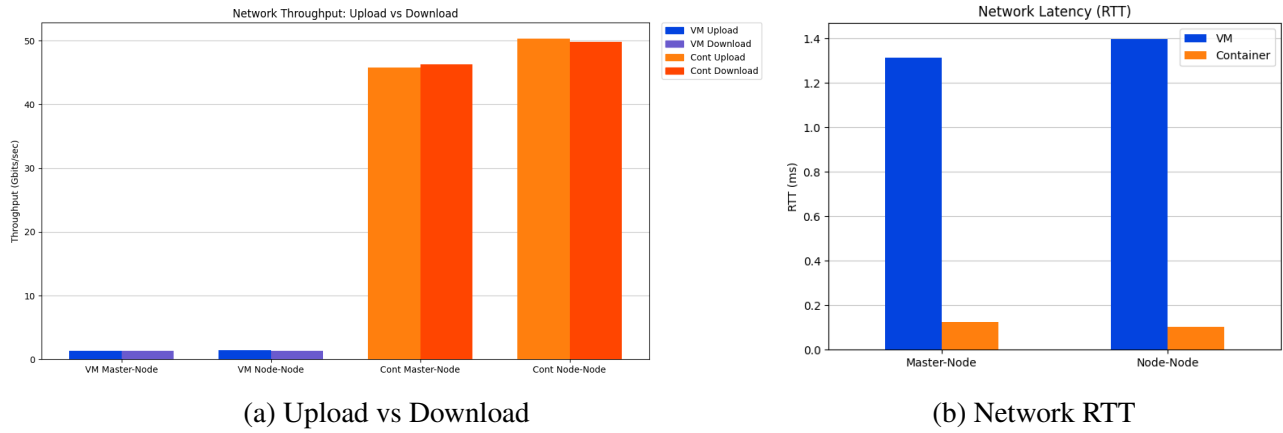


Figure 4: Network iperf3

As shown in Figure 4a, the container environment achieves massive throughput, reaching around 45 – 50 Gb/s for both master-node and node-node communication. In contrast, the virtual machines achieve only ≈ 1.5 Gb/s. This 30x performance gap highlights the substantial overhead introduced by full virtualization in the network stack of virtual machines, where packet processing must traverse additional layers such as the hypervisor and virtual switches.

Figure 4b reports Round-Trip Time (RTT) and confirms this trend. Containers again significantly outperform virtual machines, that suffer from high latency (≈ 1.35 ms) against the ≈ 0.1 ms of the containers. The reduced latency is a direct consequence of containers sharing the kernel and network stack of the underlying WSL 2 utility VM, a very light virtual machine that contains the Linux kernel on top of which Docker runs the containers. This architecture allows inter-container communication to be routed internally via the Docker bridge, without passing through the external virtual switch, effectively eliminating the packet switching delays inherent to the VM architecture.

The containers advantages are particularly relevant for distributed and communication-intensive workloads, where network performance often constitutes a critical bottleneck. The measurements confirm that, under identical resource allocations, containers enable a more efficient and predictable use of the underlying network infrastructure.

5.5 HPCC

A critical instability was observed during the HPC Challenge benchmark. While the containerized environment successfully scaled to a problem size of $N = 8192$, the VM configuration encountered system failures at problem sizes as small as $N = 512$. We present the problem with more details in Section 6.2. As a result, containers exhibited far more predictable behavior under high synchronization and communication load.

Figure 5 illustrates the HPL benchmark results for the container cluster. The configuration achieved

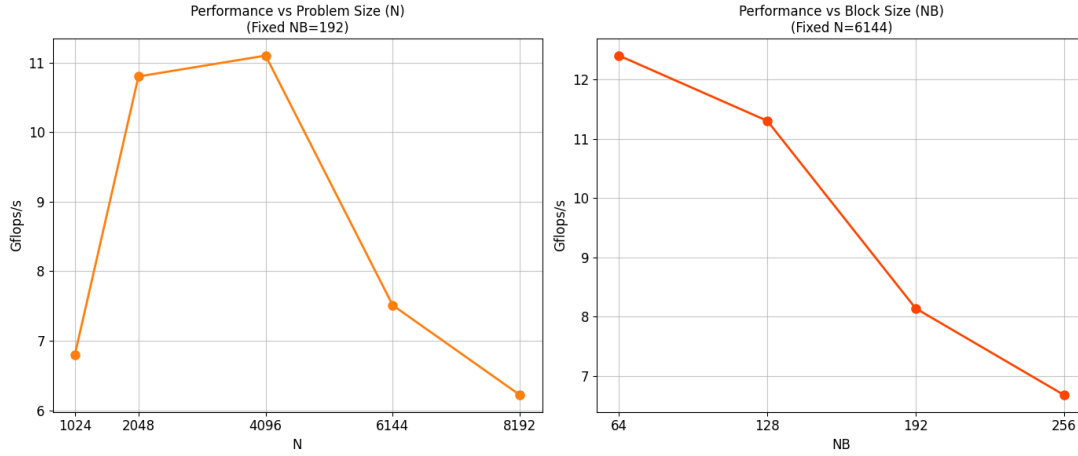


Figure 5: Containers HPCC Performance

peak performance (≈ 11.1 Gflops/s) at a problem size of $N = 4096$. Beyond this point, performance degrades. This indicates a shift from being limited by CPU speed to being limited by memory speed. Essentially, the time required for fetching data from the memory exceeds the CPU's calculation speed. The block size analysis moreover reveals a strong preference for smaller blocks ($NB = 64$). A block size of 64 corresponds to 32KB of data, which aligns perfectly with the typical L1 cache size of host processors. This allows the processor to work at peak efficiency while hiding the delays caused by the virtualized memory layer.

6 Technical Challenges

6.1 Benchmark tool selection and stress-ng exclusion

Initially, stress-ng was intended for system evaluation alongside sysbench. However, it was excluded from the final analysis.

The primary issue was a total failure of the tool's timing mechanisms. Despite configuring the tests with a `-timeout 60` parameter, the tool failed to respect the specified execution time limit. On the master node, CPU stress tests ran for approximately 7 minutes, while memory tests extended to 15 minutes. The situation was more severe on worker nodes, where CPU testing continued for over an hour until manual termination. This $60\times$ time dilation was accompanied by erratic CPU spikes and system-wide unresponsiveness, suggesting that the tool was no longer operating within valid benchmark parameters.

These symptoms are characteristic of vCPU time drift under extreme load. The stress-ng workload was so computationally intensive that it saturated the virtual CPU resources, causing the VM's internal clock to desynchronize significantly from real time. In this state, the hypervisor fails to provide ac-

curate time accounting to the guest, creating a feedback loop: severe CPU saturation induces extreme time drift, which prevents internal timeout signals from triggering, and continuous execution further perpetuates the saturation and drift.

Such behavior makes any captured metrics meaningless, as the temporal basis for "performance per second" is lost. For benchmark data to be valid, tools must be deterministic and stable. We therefore excluded `stress-ng` from our analysis and only relied on `sysbench`, which demonstrated high reliability, consistently respecting timeout parameters and providing reproducible metrics across all cluster nodes.

The `stress-ng` configuration remains in the project scripts, commented out, for reference, but its data is excluded from this report.

6.2 HPC Challenge: Stability and Virtualization Failures

During the experimental phase with the HPC Challenge, a severe system-level failure was observed despite the relatively small parameters ($N = 512$, $NB = 64$, $P = 1$, $Q = 3$). The execution was launched using OpenMPI with only three total processes of the six available:

```
1 $ mpirun.openmpi -np 3 -mca btl\__tcp\__if\__include enp0s8 --hostfile mpi-  
   hostfile --map-by ppr:1:node hpcc
```

After several minutes of execution, one of the worker nodes became unresponsive and the Linux kernel reported the following critical errors:

```
1 watchdog: BUG: soft lockup - CPU\#0 stuck for 302s  
2 rcu: INFO: rcu\__preempt self-detected stall on CPU  
3 rcu: rcu\__preempt kthread starved for 77762 jiffies  
4 Unless rcu\__preempt kthread gets sufficient CPU time, OOM is now expected  
   behavior
```

These logs indicate that the Linux kernel was unable to schedule essential internal tasks. The Read-Copy-Update (RCU) subsystem, responsible for memory consistency, was starved of CPU time, leading to a total system freeze.

The failure did not originate from the benchmark size itself, which was intentionally kept small. Instead, the root cause probably lies in the interaction between intensive MPI collective communication, memory allocation and synchronization pressure generated by HPCC, and the scheduling and interrupt handling mechanisms of the hypervisor. This pushed the system beyond its limits and repeatedly triggered the Linux kernel's Out-Of-Memory (OOM) killer, which terminated MPI processes to protect the host, resulting in aborted jobs.

RCU stalls and watchdog timeouts are also a classic symptom of the "Lock Holder Preemption" phenomenon. In a standard VM, if the host scheduler pauses a virtual CPU while it is holding a critical kernel lock, other vCPUs inside the VM will "spin" indefinitely waiting for that lock to be released.

Since the VM cannot force the host to wake up the lock-holder, the internal watchdog timer expires, assuming the system has crashed.

For this reason, stable and reproducible measurements were obtained only in single-node configurations, which avoided the distributed initialization overhead of MPI. In any case, the host machine had approximately 7.7 GB of total memory, of which around 80% was already in use at runtime. The limitations observed therefore reflect the physical constraints of the experimental platform rather than flaws in the benchmark software or in the cluster configuration itself.

7 Conclusion

This comparative study highlights the significant performance and stability advantages of containerized environments over traditional virtual machines for parallel tasks when resources are limited.

Our results consistently showed that containers outperform VMs in raw computational tasks and disk I/O. In sequential and random disk tests, containers achieved nearly double the throughput of VMs, while inter-node latency was an order of magnitude lower (≈ 0.1 ms vs. ≈ 1.35 ms). This is primarily due to the Container's ability to share the underlying WSL 2 utility kernel and network stack, avoiding the heavy packet-switching overhead inherent to standard hypervisors.

A critical finding of this project was the stability gap between the two technologies. Under the intense pressure of the HPC Challenge and stress-ng benchmarks, the virtual machine cluster suffered from catastrophic failures, including RCU stalls and watchdog timeouts. These failures were traced to "Lock Holder Preemption" and vCPU time drift-issues that arise when a hypervisor cannot effectively synchronize a guest OS under full load. Conversely, the container environment remained stable, successfully scaling to problem sizes ($N = 8192$) that were sixteen times larger than the failure threshold of the VMs ($N = 512$).

However, the analysis also revealed nuances in filesystem performance. While containers won on raw block I/O, VMs proved competitive in reading and writing files using standard libraries. This suggests that the dedicated, static memory allocation of a VM can, in specific cases, provide more efficient userspace buffering than the dynamic memory scaling of WSL 2.

In summary, for users deploying distributed clusters on local hardware, containers offer a more robust and efficient solution. They not only provide higher performance but also exhibit a resilience to high-load scheduling conflicts that virtual machines cannot match without significant tuning.

List of Figures

1	CPU sysbench	18
2	Memory sysbench	19
3	Disk I/O IOZone	20
4	Network iperf3	21
5	Containers HPCC Performance	22

Nomenclature

- BTL Byte Transfer Layer
- DHPC Dynamic Host Configuration Protocol
- HPC High Performance Computing
- I/O Input/Output
- ICMP Internet Control Message Protocol
- MPI Message Passing Interface
- NAT Network Address Translation
- NFS Network File System
- OOM Out Of Memory
- OS Operating System
- RCU Read-Copy-Update
- RTT Round-Trip Time
- SSH Secure Shell
- TCP Transmission Control Protocol
- VM Virtual Machine
- WSL Windows Subsystem for Linux

References

- [1] Canonical Ltd., “How to Install and Configure NFS on Ubuntu 22.04.” <https://documentation.ubuntu.com/server/how-to/networking/install-nfs/>, 2024. Accessed: 2024-05-20.
- [2] The Linux Kernel Archives, “NFS Administration Guide.” <https://docs.kernel.org/admin-guide/nfs/index.html>, 2024. Official Linux Kernel Documentation.
- [3] Netplan Project, “Netplan Reference and Tutorial.” <https://netplan.readthedocs.io/>, 2024. Covers YAML configuration and tutorial guides.
- [4] S. Kelley, “Dnsmasq Documentation and Man Page.” <https://dnsmasq.org/docs/dnsmasq-man.html>, 2024. Lightweight DNS and DHCP server documentation.
- [5] A. Kopytov, “sysbench(1) Multi-threaded benchmark tool.” <https://manpages.debian.org/testing/sysbench/sysbench.1.en.html>, 2024.
- [6] ESnet, “iPerf3 User Documentation.” <https://iperf.fr/iperf-doc.php>, 2024. The TCP, UDP and SCTP network bandwidth measurement tool.
- [7] A. Petitet, R. C. Whaley, J. Dongarra, and A. Cleary, “HPL - A Portable Implementation of the High-Performance Linpack Benchmark.” <https://www.netlib.org/benchmark/hpl/>, 2018. Official HPL reference implementation.
- [8] Advanced Clustering Technologies, “HPL Tuning and HPL.dat Configuration.” https://www.advancedclustering.com/act_kb/tune-hpl-dat-file/, 2024. Technical knowledge base for Linpack optimization.